Seminar 3 - Frontend cu React, Axios si Tailwind CSS

Cuprins

1. Vite, React si Tailwind CSS
2. Componente, JSX si Props
3. State si evenimente (`useState`)
4. Efecte si consumul API-urilor cu Axios (`useEffect`)
5. Rutare in aplicatie (`react-router-dom`)
6. Paginile de autentificare: Register si Login
7. Stare globala de autentificare (`AuthContext`) si rute protejate
8. Catalogul de produse: listare, filtrare si detalii
9. Structura finala si checklist

Exemplele din seminar continua aplicatia de **magazin online** din Seminar 2. Vom construi interfata web in **React (doar JSX, fara TypeScript)**, stilizata simplu cu **Tailwind CSS**, care comunica prin **Axios** cu API-ul Express pornit pe `http://localhost:3000`.

Cerinte preliminare

- Node.js versiunea 20 sau mai noua (`node -v`)
- un editor de cod (VS Code)
- **backend-ul din Seminar 2 pornit** pe `http://localhost:3000` (`npm run dev` in `ecommerce-api`) si baza populata cu `npx prisma db seed`

1. Vite, React si Tailwind CSS

React este o biblioteca JavaScript pentru construirea interfetelor grafice din **componente** reutilizabile. In loc sa modificam manual DOM-ul (`document.querySelector(...)`), descriem cum arata interfata in functie de datele curente (**state**), iar React actualizeaza automat doar elementele modificate.

Vite este instrumentul de build si serverul de dezvoltare care ruleaza aplicatia React local (implicit pe `http://localhost:5173`).

Crearea proiectului

Intr-un folder separat fata de `ecommerce-api`, cream proiectul React cu template-ul JavaScript (JSX):

```
npm create vite@latest ecommerce-web -- --template react
cd ecommerce-web
npm install
```

Dependente

Instalam biblioteca pentru request-uri HTTP (`axios`), router-ul pentru navigarea intre pagini (`react-router-dom`) si **Tailwind CSS**:

```
npm install axios react-router-dom
npm install -D tailwindcss @tailwindcss/vite
```

Pachet	Rol
`react`, `react-dom`	nucleul React si randarea in browser
`axios`	client HTTP pentru apelurile catre API-ul din Seminar 2
`react-router-dom`	navigare intre pagini fara reincarcarea browserului (SPA)
`tailwindcss`, `@tailwindcss/vite`	clase utilitare CSS pentru stilizare rapida

Configurarea Tailwind CSS

Adaugam plugin-ul Tailwind in fisierul `vite.config.js`:

```
import { defineConfig } from "vite";
import react from "@vitejs/plugin-react";
import tailwindcss from "@tailwindcss/vite";

export default defineConfig({
```

```
    plugins: [react(), tailwindcss()]
});
```

Inlocuim continutul fisierului `src/index.css` cu o singura linie:

```
@import "tailwindcss";
```

Pornire:

```
npm run dev
```

Aplicatia va fi disponibila la `http://localhost:5173` — exact adresa configurata in `cors({ origin: "http://localhost:5173", credentials: true })` pe serverul din Seminar 2.

Structura proiectului

```
ecommerce-web/
├── index.html
├── vite.config.js
├── package.json
└── src/
    ├── main.jsx
    ├── index.css
    ├── App.jsx
    ├── api/
    │   └── client.js
    ├── context/
    │   └── AuthContext.jsx
    ├── components/
    │   ├── Navbar.jsx
    │   ├── ProductCard.jsx
    │   └── ProtectedRoute.jsx
    └── pages/
        ├── RegisterPage.jsx
        ├── LoginPage.jsx
        ├── ProductsPage.jsx
        └── ProductDetailPage.jsx
```

Cream folderele:

```
mkdir -p src/api src/context src/components src/pages
```

2. Componente, JSX si Props

O **componenta** React este o functie JavaScript al carei nume incepe cu **litera mare** si care returneaza elemente **JSX** (o extensie de sintaxa ce permite scrierea de cod asemanator HTML direct in JavaScript).

Reguli de baza in JSX

HTML clasic	JSX	Motiv
`class="card"`	`className="card"`	`class` este cuvant rezervat in JavaScript
`for="email"`	`htmlFor="email"`	`for` este cuvant rezervat in JavaScript
`<img src="...">`	`<img src="..." />`	toate tag-urile trebuie inchise
text static	`{expresie}`	acoladele evalueaza orice expresie JavaScript

Props si compunere

Datele se transmit de la o componenta parinte la o componenta copil prin **props** (proprietati), primite ca un obiect (de obicei destructurat, ca in Seminar 1):

```
// src/components/ProductCard.jsx
import { Link } from "react-router-dom";

export default function ProductCard({ product }) {
    const inStock = product.stock > 0;

    return (
        <div className="bg-white rounded-xl border border-gray-200 shadow-sm
overflow-hidden flex flex-col">
            <img
                src={product.imageUrl || "https://picsum.photos/400/300"}
                alt={product.name}
                className="w-full h-44 object-cover bg-gray-100"
            />

            <div className="p-4 flex flex-col flex-1">
                <div className="flex items-center justify-between gap-2 mb-1">
                    <span className="text-xs font-medium text-blue-600 bg-blue-50
px-2 py-0.5 rounded">
                        {product.category}
                    </span>
                    <span className={`text-xs font-medium ${inStock ? "text-green-
600" : "text-red-500"}`}>
                        {inStock ? `In stoc (${product.stock})` : "Stoc epuizat"}
                    </span>
                </div>

                <h3 className="text-lg font-semibold text-gray-900">
```

```
                    {product.name}
                </h3>

                <p className="text-sm text-gray-500 line-clamp-2 mt-1 flex-1">
                    {product.description}
                </p>

                <div className="mt-4 pt-3 border-t border-gray-100 flex items-
center justify-between">
                    <span className="text-lg font-bold text-gray-900">
                        {product.price} RON
                    </span>

                    <Link
                        to={`/products/${product.id}`}
                        className="text-sm font-medium text-white bg-blue-600
hover:bg-blue-700 px-3 py-1.5 rounded-lg"
                    >
                        Detalii
                    </Link>
                </div>
            </div>
        </div>
    );
}
```

Observati:

- randarea conditionala cu operatorul ternar (`inStock ? ... : ...`);
- afisarea listelor in React se face cu `.map()`, iar fiecare element din lista trebuie sa aiba un atribut unic `key`:

```
{products.map(product => (
    <ProductCard key={product.id} product={product} />
))}
```

3. State si evenimente (`useState`)

Variabilele locale obisnuite din interiorul unei componente nu declanseaza actualizarea ecranului cand isi schimba valoarea. Pentru date care se modifica in timp, folosim hook-ul `useState`.

```
import { useState } from "react";

const [count, setCount] = useState(0);
```

- `count` este valoarea curenta a starii;
- `setCount` este functia care actualizeaza starea si cere React-ului sa **re-randeze** componenta;
- starea nu se modifica niciodata direct (`count = 5` nu functioneaza).

Evenimente si formulare controlate

In React, evenimentele se scriu in camelCase (`onClick`, `onChange`, `onSubmit`). Pentru input-uri, folosim **controlled components**: valoarea input-ului vine din state, iar `onChange` actualizeaza state-ul la fiecare tasta apasata.

```
import { useState } from "react";

export default function App() {
    const [query, setQuery] = useState("");

    const handleSubmit = (e) => {
        e.preventDefault(); // opreste reincarcarea paginii de catre browser
        console.log("Cautare:", query);
    };

    return (
        <form onSubmit={handleSubmit} className="flex gap-2">
            <input
                type="text"
                value={query}
                onChange={(e) => setQuery(e.target.value)}
                placeholder="Cauta produs..."
                className="rounded-lg border border-gray-300 px-3 py-2 text-sm"
            />
            <button
                type="submit"
                className="bg-blue-600 text-white px-4 py-2 rounded-lg text-sm"
            >
                Cauta
            </button>
        </form>
```

```
    );
}
```

> **Important:** La orice formular (`<form onSubmit={...}>`), apelati intotdeauna `e.preventDefault()`. Altfel browserul face un request HTTP clasic si reincarca toata pagina.

4. Efecte si consumul API-urilor cu Axios (`useEffect`)

Configurarea instantei Axios

In Seminar 2, autentificarea foloseste un **cookie HttpOnly** (`token`). Pentru ca browserul sa accepte cookie-ul la `/api/auth/login` si sa il trimita automat la request-urile ulterioare catre `localhost:3000`, Axios trebuie configurat cu `withCredentials: true`.

Cream o singura instanta centralizata in fisierul `src/api/client.js`:

```
import axios from "axios";

export const api = axios.create({
    baseURL: "http://localhost:3000/api",
    withCredentials: true
});
```

Utilizare in componente:

```
// echivalent cu GET http://localhost:3000/api/products
const res = await api.get("/products");
console.log(res.data); // corpul JSON returnat de Express
```

Tratarea erorilor din API

In Seminar 2 (sectiunea 3), am standardizat toate erorile din backend in formatul:

```
{ "message": "Produsul nu exista" }
```

Cand serverul raspunde cu status `4xx` sau `5xx`, Axios arunca o exceptie. Mesajul trimis de backend se afla in `err.response?.data?.message`:

```
try {
    await api.post("/auth/login", { email, password });
} catch (err) {
    const message = err.response?.data?.message || "Eroare de conexiune cu
serverul";
    setError(message);
}
```

Hook-ul `useEffect`

`useEffect` ruleaza cod cu efecte secundare (cum ar fi un apel HTTP la incarcarea paginii) **dupa** ce componenta a fost randata.

```
import { useEffect, useState } from "react";
import { api } from "../api/client";

useEffect(() => {
    const fetchProducts = async () => {
        try {
            setLoading(true);
            setError("");
            const res = await api.get("/products");
            setProducts(res.data);
        } catch (err) {
            setError(err.response?.data?.message || "Nu s-au putut incarca
produsele");
        } finally {
            setLoading(false);
        }
    };

    fetchProducts();
}, []);
```

Al doilea argument al lui `useEffect` este **lista de dependente**:

Dependenta	Cand ruleaza efectul
`[]` (array gol)	**o singura data**, la montarea componentei
`[category, sort]`	la montare si ori de cate ori se schimba `category` sau `sort`
lipseste	la **fiecare** randare (evitati — produce bucle infinite daca apelati `setState`!)

> Functia transmisa direct lui `useEffect` **nu** poate fi `async` (`useEffect(async () => ...)` este gresit). Definim o functie `async` in interiorul ei si o apelam imediat.

5. Rutare in aplicatie (`react-router-dom`)

O aplicatie React este un **SPA** (Single Page Application): navigarea intre pagini nu cere fisiere HTML noi de la server, ci schimba componenta afisata in functie de URL.

Element	Rol
`BrowserRouter`	activeaza rutarea bazata pe URL-ul browserului
`Routes` / `Route`	asociaza o cale (`path`) cu o componenta (`element`)
`Link`	navigare declarativa intre pagini (inlocuieste `<a href="...">`)
`useNavigate()`	navigare programatica din cod (ex.: dupa login reusit)
`useParams()`	citeste parametrii dinamici din URL (ex.: `/products/:id`)

Bara de navigare (`Navbar.jsx`)

Fisierul `src/components/Navbar.jsx` (il vom completa in sectiunea 7 cu starea utilizatorului logat):

```
import { Link } from "react-router-dom";

export default function Navbar() {
    return (
        <header className="bg-white border-b border-gray-200">
            <div className="max-w-6xl mx-auto px-4 h-16 flex items-center justify-
between">
                <Link to="/products" className="text-xl font-bold text-blue-600">
                    ShopApp
                </Link>

                <nav className="flex items-center gap-4 text-sm font-medium">
                    <Link to="/products" className="text-gray-700 hover:text-blue-
600">
                        Produse
                    </Link>
                    <Link to="/login" className="text-gray-700 hover:text-blue-
600">
                        Autentificare
                    </Link>
                    <Link
                        to="/register"
                        className="bg-blue-600 text-white px-3 py-1.5 rounded-lg
hover:bg-blue-700"
                    >
                        Inregistrare
                    </Link>
                </nav>
            </div>
        </header>
```

```
    );
}
```

Configurarea rutelor in `App.jsx`

```
import { BrowserRouter, Routes, Route, Navigate } from "react-router-dom";
import Navbar from "./components/Navbar";
import ProductsPage from "./pages/ProductsPage";

export default function App() {
    return (
        <BrowserRouter>
            <div className="min-h-screen bg-gray-50 text-gray-800">
                <Navbar />
                <main className="max-w-6xl mx-auto px-4 py-8">
                    <Routes>
                        <Route path="/" element={<Navigate to="/products" replace
/>} />
                        <Route path="/products" element={<ProductsPage />} />
                    </Routes>
                </main>
            </div>
        </BrowserRouter>
    );
}
```

6. Paginile de autentificare: Register si Login

In Seminar 2 am implementat urmatoarele rute:

- `POST /api/auth/register`: primeste `{ name, email, password }` si returneaza `201` (fara corp si fara cookie);
- `POST /api/auth/login`: primeste `{ email, password }`, seteaza cookie-ul `token` (`HttpOnly`) si returneaza `{ user }`.

Pagina de inregistrare (`RegisterPage.jsx`)

Fisierul `src/pages/RegisterPage.jsx`:

```
import { useState } from "react";
import { Link, useNavigate } from "react-router-dom";
import { api } from "../api/client";

export default function RegisterPage() {
    const navigate = useNavigate();

    const [name, setName] = useState("");
    const [email, setEmail] = useState("");
    const [password, setPassword] = useState("");
    const [error, setError] = useState("");
    const [submitting, setSubmitting] = useState(false);

    const handleSubmit = async (e) => {
        e.preventDefault();
        setError("");

        if (password.length < 6) {
            setError("Parola trebuie sa aiba minim 6 caractere");
            return;
        }

        try {
            setSubmitting(true);
            await api.post("/auth/register", { name, email, password });
            navigate("/login", {
                state: { message: "Cont creat cu succes! Te poti autentifica." }
            });
        } catch (err) {
            setError(err.response?.data?.message || "Eroare la inregistrare");
        } finally {
            setSubmitting(false);
        }
    };

    return (
        <div className="max-w-md mx-auto bg-white rounded-xl border border-gray-
```

```
200 shadow-sm p-6">
            <h1 className="text-2xl font-bold text-gray-900 mb-1">Creare cont</h1>
            <p className="text-sm text-gray-500 mb-6">
                Completeaza datele pentru a-ti crea un cont nou.
            </p>

            {error && (
                <div className="mb-4 rounded-lg bg-red-50 border border-red-200
px-3 py-2 text-sm text-red-700">
                    {error}
                </div>
            )}

            <form onSubmit={handleSubmit} className="space-y-4">
                <div>
                    <label htmlFor="name" className="block text-sm font-medium
text-gray-700 mb-1">
                        Nume
                    </label>
                    <input
                        id="name"
                        type="text"
                        required
                        value={name}
                        onChange={(e) => setName(e.target.value)}
                        placeholder="Ana Popescu"
                        className="w-full rounded-lg border border-gray-300 px-3
py-2 text-sm focus:border-blue-500 focus:outline-none"
                    />
                </div>

                <div>
                    <label htmlFor="email" className="block text-sm font-medium
text-gray-700 mb-1">
                        Email
                    </label>
                    <input
                        id="email"
                        type="email"
                        required
                        value={email}
                        onChange={(e) => setEmail(e.target.value)}
                        placeholder="ana@test.com"
                        className="w-full rounded-lg border border-gray-300 px-3
py-2 text-sm focus:border-blue-500 focus:outline-none"
                    />
                </div>

                <div>
                    <label htmlFor="password" className="block text-sm font-medium
text-gray-700 mb-1">
                        Parola
                    </label>
                    <input
```

```
                        id="password"
                        type="password"
                        required
                        value={password}
                        onChange={(e) => setPassword(e.target.value)}
                        placeholder="Minim 6 caractere"
                        className="w-full rounded-lg border border-gray-300 px-3
py-2 text-sm focus:border-blue-500 focus:outline-none"
                    />
                </div>

                <button
                    type="submit"
                    disabled={submitting}
                    className="w-full bg-blue-600 text-white font-medium py-2
rounded-lg text-sm hover:bg-blue-700 disabled:opacity-50 cursor-pointer"
                >
                    {submitting ? "Se creeaza contul..." : "Inregistreaza-te"}
                </button>
            </form>

            <p className="mt-4 text-center text-sm text-gray-600">
                Ai deja cont?{" "}
                <Link to="/login" className="text-blue-600 font-medium
hover:underline">
                    Autentifica-te
                </Link>
            </p>
        </div>
    );
}
```

Pagina de autentificare (`LoginPage.jsx`)

Fisierul `src/pages/LoginPage.jsx`:

```
import { useState } from "react";
import { Link, useLocation, useNavigate } from "react-router-dom";
import { useAuth } from "../context/AuthContext";

export default function LoginPage() {
    const navigate = useNavigate();
    const location = useLocation();
    const { login } = useAuth();

    const successMessage = location.state?.message || "";

    const [email, setEmail] = useState("");
    const [password, setPassword] = useState("");
    const [error, setError] = useState("");
    const [submitting, setSubmitting] = useState(false);
```

```
    const handleSubmit = async (e) => {
        e.preventDefault();
        setError("");

        try {
            setSubmitting(true);
            await login(email, password);
            navigate("/products");
        } catch (err) {
            setError(err.response?.data?.message || "Eroare tehnica. Incearca mai
tarziu.");
        } finally {
            setSubmitting(false);
        }
    };

    return (
        <div className="max-w-md mx-auto bg-white rounded-xl border border-gray-
200 shadow-sm p-6">
            <h1 className="text-2xl font-bold text-gray-900 mb-
1">Autentificare</h1>
            <p className="text-sm text-gray-500 mb-6">
                Intra in contul tau pentru a comanda produse.
            </p>

            {successMessage && (
                <div className="mb-4 rounded-lg bg-green-50 border border-green-
200 px-3 py-2 text-sm text-green-700">
                    {successMessage}
                </div>
            )}

            {error && (
                <div className="mb-4 rounded-lg bg-red-50 border border-red-200
px-3 py-2 text-sm text-red-700">
                    {error}
                </div>
            )}

            <form onSubmit={handleSubmit} className="space-y-4">
                <div>
                    <label htmlFor="email" className="block text-sm font-medium
text-gray-700 mb-1">
                        Email
                    </label>
                    <input
                        id="email"
                        type="email"
                        required
                        value={email}
                        onChange={(e) => setEmail(e.target.value)}
                        placeholder="admin@shop.com"
                        className="w-full rounded-lg border border-gray-300 px-3
```

```
py-2 text-sm focus:border-blue-500 focus:outline-none"
                    />
                </div>

                <div>
                    <label htmlFor="password" className="block text-sm font-medium
text-gray-700 mb-1">
                        Parola
                    </label>
                    <input
                        id="password"
                        type="password"
                        required
                        value={password}
                        onChange={(e) => setPassword(e.target.value)}
                        placeholder="••••••••"
                        className="w-full rounded-lg border border-gray-300 px-3
py-2 text-sm focus:border-blue-500 focus:outline-none"
                    />
                </div>

                <button
                    type="submit"
                    disabled={submitting}
                    className="w-full bg-blue-600 text-white font-medium py-2
rounded-lg text-sm hover:bg-blue-700 disabled:opacity-50  cursor-pointer"
                >
                    {submitting ? "Se autentifica..." : "Intra in cont"}
                </button>
            </form>

            <p className="mt-4 text-center text-sm text-gray-600">
                Nu ai cont?{" "}
                <Link to="/register" className="text-blue-600 font-medium
hover:underline">
                    Creeaza unul
                </Link>
            </p>
        </div>
    );
}
```

> Puteti testa imediat cu contul creat in seed-ul din Seminar 2: `admin@shop.com` / `admin123`.

7. Stare globala de autentificare (`AuthContext`) si rute protejate

Mai multe componente au nevoie sa stie cine este utilizatorul logat (`Navbar`, `LoginPage`, paginile de comenzi). In loc sa pasam `user` prin props din componenta in componenta, folosim **React Context**.

In plus, cand utilizatorul da **Refresh** la pagina, starea React din memorie se reseteaza, dar **cookie-ul HttpOnly ramane in browser**. De aceea, la pornirea aplicatiei apelam `GET /api/auth/me` pentru a recupera datele utilizatorului curent.

Crearea `AuthContext.jsx`

Fisierul `src/context/AuthContext.jsx`:

```
import { createContext, useContext, useEffect, useState } from "react";
import { api } from "../api/client";

const AuthContext = createContext(null);

export function AuthProvider({ children }) {
    const [user, setUser] = useState(null);
    const [loading, setLoading] = useState(true);

    useEffect(() => {
        const loadUser = async () => {
            try {
                const res = await api.get("/auth/me");
                setUser(res.data);
            } catch {
                setUser(null);
            } finally {
                setLoading(false);
            }
        };

        loadUser();
    }, []);

    const login = async (email, password) => {
        const res = await api.post("/auth/login", { email, password });
        setUser(res.data.user);
    };

    const logout = async () => {
        await api.post("/auth/logout");
        setUser(null);
    };

    return (
```

```
            <AuthContext.Provider value={{ user, loading, login, logout }}>
                {children}
            </AuthContext.Provider>
        );
}

export const useAuth = () => useContext(AuthContext);
```

In `App.jsx` invelim aplicatia in `<AuthProvider>`:

```
// in App.jsx
<AuthProvider>
            {/* ... Navbar si Routes ... */}
</AuthProvider>
```

Actualizarea `Navbar.jsx`

Acum bara de navigare afiseaza numele utilizatorului si butonul **Logout** atunci cand este autentificat:

```
import { Link, useNavigate } from "react-router-dom";
import { useAuth } from "../context/AuthContext";

export default function Navbar() {
    const { user, logout } = useAuth();
    const navigate = useNavigate();

    const handleLogout = async () => {
        await logout();
        navigate("/login");
    };

    return (
        <header className="bg-white border-b border-gray-200">
            <div className="max-w-6xl mx-auto px-4 h-16 flex items-center justify-
between">
                <Link to="/products" className="text-xl font-bold text-blue-600">
                    ShopApp
                </Link>

                <nav className="flex items-center gap-4 text-sm font-medium">
                    <Link to="/products" className="text-gray-700 hover:text-blue-
600">
                        Produse
                    </Link>

                    {user ? (
                        <>
                            <span className="text-gray-500">
                                Salut, <strong className="text-gray-800">
```

```
{user.name}</strong>
                            </span>
                            <button
                                onClick={handleLogout}
                                className="text-red-600 hover:text-red-700 font-
medium cursor-pointer"
                            >
                                Logout
                            </button>
                        </>
                    ) : (
                        <>
                            <Link to="/login" className="text-gray-700 hover:text-
blue-600">
                                Autentificare
                            </Link>
                            <Link
                                to="/register"
                                className="bg-blue-600 text-white px-3 py-1.5
rounded-lg hover:bg-blue-700"
                            >
                                Inregistrare
                            </Link>
                        </>
                    )}
                </nav>
            </div>
        </header>
    );
}
```

Rute protejate (`ProtectedRoute.jsx`)

Pentru paginile care necesita autentificare (ex.: cosul de cumparaturi sau comenzile din exercitii), cream un wrapper care redirectioneaza utilizatorii nelogati catre `/login`.

Fisierul `src/components/ProtectedRoute.jsx`:

```
import { Navigate } from "react-router-dom";
import { useAuth } from "../context/AuthContext";

export default function ProtectedRoute({ children }) {
    const { user, loading } = useAuth();

    if (loading) {
        return <p className="text-center text-gray-500 py-8">Se verifica
sesiunea...</p>;
    }

    if (!user) {
        return <Navigate to="/login" replace />;
```

```
    }

    return children;
}
```

In `App.jsx` invelim aplicatia in `<BrowserRouter>`:

```
// in App.jsx
<AuthProvider>
    <BrowserRouter>
        {/* ... Navbar si Routes ... */}
    </BrowserRouter>
</AuthProvider>
```

8. Catalogul de produse: listare, filtrare si detalii

In Seminar 2 si in Exercitiile din Seminar 2 am construit urmatoarele endpoint-uri:

- `GET /api/products?search=...&category=...&sort=...` — returneaza un array de produse;
- `GET /api/products/categories` — returneaza lista categoriilor distincte;
- `GET /api/products/:id` — returneaza detaliile unui singur produs.

Pagina de produse (`ProductsPage.jsx`)

In Axios, obiectul `params` transforma automat campurile in query string (ex.: `{ category: "Electronics", sort: "price_asc" }` devine `?category=Electronics&sort=price_asc`). Parametrii cu valoarea `undefined` sunt ignorati automat de Axios.

Fisierul `src/pages/ProductsPage.jsx`:

```
import { useEffect, useState } from "react";
import { api } from "../api/client";
import ProductCard from "../components/ProductCard";

export default function ProductsPage() {
    const [products, setProducts] = useState([]);
    const [categories, setCategories] = useState([]);
    const [search, setSearch] = useState("");
    const [category, setCategory] = useState("");
    const [sort, setSort] = useState("newest");
    const [loading, setLoading] = useState(true);
    const [error, setError] = useState("");

    useEffect(() => {
        api.get("/products/categories")
            .then((res) => setCategories(res.data))
            .catch(() => setCategories(["Electronics", "Furniture"]));
    }, []);

    useEffect(() => {
        const fetchProducts = async () => {
            try {
                setLoading(true);
                setError("");

                const res = await api.get("/products", {
                    params: {
                        search: search.trim() || undefined,
                        category: category || undefined,
                        sort
                    }
                });

                setProducts(res.data);
```

```
            } catch (err) {
                setError(err.response?.data?.message || "Eroare la incarcarea
produselor");
            } finally {
                setLoading(false);
            }
        };

        fetchProducts();
    }, [search, category, sort]);

    return (
        <div>
            <div className="mb-6">
                <h1 className="text-2xl font-bold text-gray-900">Catalog
produse</h1>
                <p className="text-sm text-gray-500">
                    Exploreaza produsele disponibile in magazin.
                </p>
            </div>

            {/* Bara de cautare, filtrare si sortare */}
            <div className="bg-white p-4 rounded-xl border border-gray-200 shadow-
sm mb-6 grid grid-cols-1 sm:grid-cols-3 gap-4">
                <input
                    type="text"
                    value={search}
                    onChange={(e) => setSearch(e.target.value)}
                    placeholder="Cauta dupa nume..."
                    className="rounded-lg border border-gray-300 px-3 py-2 text-sm
focus:border-blue-500 focus:outline-none"
                />

                <select
                    value={category}
                    onChange={(e) => setCategory(e.target.value)}
                    className="rounded-lg border border-gray-300 px-3 py-2 text-sm
bg-white focus:border-blue-500 focus:outline-none"
                >
                    <option value="">Toate categoriile</option>
                    {categories.map((cat) => (
                        <option key={cat} value={cat}>
                            {cat}
                        </option>
                    ))}
                </select>

                <select
                    value={sort}
                    onChange={(e) => setSort(e.target.value)}
                    className="rounded-lg border border-gray-300 px-3 py-2 text-sm
bg-white focus:border-blue-500 focus:outline-none"
                >
                    <option value="newest">Cele mai noi</option>
```

```
                    <option value="price_asc">Pret: crescator</option>
                    <option value="price_desc">Pret: descrescator</option>
                    <option value="name">Nume: A - Z</option>
                </select>
            </div>

            {error && (
                <div className="mb-6 rounded-lg bg-red-50 border border-red-200
px-4 py-3 text-sm text-red-700">
                    {error}
                </div>
            )}

            {loading ? (
                <p className="text-center text-gray-500 py-12">Se incarca
produsele...</p>
            ) : products.length === 0 ? (
                <div className="bg-white rounded-xl border border-gray-200 p-8
text-center text-gray-500">
                    Nu a fost gasit niciun produs pentru filtrele selectate.
                </div>
            ) : (
                <div className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-3
gap-6">
                    {products.map((product) => (
                        <ProductCard key={product.id} product={product} />
                    ))}
                </div>
            )}
        </div>
    );
}
```

Pagina de detalii produs (`ProductDetailPage.jsx`)

Citeste `id`-ul produsului din URL cu `useParams()` si apeleaza `GET /api/products/:id`.

Fisierul `src/pages/ProductDetailPage.jsx`:

```
import { useEffect, useState } from "react";
import { Link, useParams } from "react-router-dom";
import { api } from "../api/client";

export default function ProductDetailPage() {
    const { id } = useParams();

    const [product, setProduct] = useState(null);
    const [loading, setLoading] = useState(true);
    const [error, setError] = useState("");

    useEffect(() => {
```

```
        const fetchProduct = async () => {
            try {
                setLoading(true);
                setError("");
                const res = await api.get(`/products/${id}`);
                setProduct(res.data);
            } catch (err) {
                setError(err.response?.data?.message || "Produsul nu a putut fi
incarcat");
            } finally {
                setLoading(false);
            }
        };

        fetchProduct();
    }, [id]);

    if (loading) {
        return <p className="text-center text-gray-500 py-12">Se incarca
detaliile...</p>;
    }

    if (error) {
        return (
            <div className="max-w-lg mx-auto bg-white rounded-xl border border-
red-200 p-6 text-center">
                <p className="text-red-600 font-medium mb-4">{error}</p>
                <Link to="/products" className="text-sm text-blue-600
hover:underline">
                    Inapoi la produse
                </Link>
            </div>
        );
    }

    const inStock = product.stock > 0;

    return (
        <div className="max-w-4xl mx-auto">
            <Link
                to="/products"
                className="inline-block text-sm text-gray-600 hover:text-blue-600
mb-4"
            >
                &larr; Inapoi la catalog
            </Link>

            <div className="bg-white rounded-xl border border-gray-200 shadow-sm
overflow-hidden grid grid-cols-1 md:grid-cols-2">
                <img
                    src={product.imageUrl || "https://picsum.photos/600/450"}
                    alt={product.name}
                    className="w-full h-72 md:h-full object-cover bg-gray-100"
                />
```

```
                <div className="p-6 flex flex-col justify-between">
                    <div>
                        <span className="text-xs font-medium text-blue-600 bg-
blue-50 px-2.5 py-1 rounded">
                            {product.category}
                        </span>

                        <h1 className="text-2xl font-bold text-gray-900 mt-3">
                            {product.name}
                        </h1>

                        <p className="text-2xl font-bold text-blue-600 mt-2">
                            {product.price} RON
                        </p>

                        <p className="text-sm text-gray-600 mt-4 leading-relaxed">
                            {product.description || "Fara descriere disponibila."}
                        </p>
                    </div>

                    <div className="mt-6 pt-4 border-t border-gray-100">
                        <p className={`text-sm font-medium ${inStock ? "text-
green-600" : "text-red-500"}`}>
                            {inStock
                                ? `Disponibil in stoc: ${product.stock} buc.`
                                : "Momentan indisponibil"}
                        </p>
                    </div>
                </div>
            </div>
        </div>
    );
}
```

9. Structura finala si checklist

Paginile aplicatiei si endpoint-urile consumate

Ruta React	Componenta	Endpoint API apelat	Acces
`/register`	`RegisterPage`	`POST /api/auth/register`	public
`/login`	`LoginPage`	`POST /api/auth/login`	public
`/products`	`ProductsPage`	`GET /api/products`, `GET /api/products/categories`	public
`/products/:id`	`ProductDetailPage`	`GET /api/products/:id`	public
(global)	`AuthContext`	`GET /api/auth/me`, `POST /api/auth/logout`	automat

Checklist

Inainte de a trece la exercitii, verificati ca:

- ☐ serverul din Seminar 2 ruleaza pe `http://localhost:3000`, iar aplicatia React pe `http://localhost:5173`;
- ☐ pe `/register` puteti crea un utilizator nou si sunteti redirectionat catre `/login` cu mesaj de confirmare;
- ☐ la incercarea de inregistrare cu un email existent se afiseaza mesajul de eroare de la backend (`409`);
- ☐ pe `/login` autentificarea cu `admin@shop.com` / `admin123` seteaza cookie-ul `token` in DevTools → Application → Cookies;
- ☐ la refresh (`F5`) pe orice pagina, utilizatorul ramane autentificat (datorita apelului `GET /api/auth/me`);
- ☐ apasarea butonului **Logout** apeleaza `POST /api/auth/logout`, sterge cookie-ul si actualizeaza `Navbar`;
- ☐ pe `/products` se afiseaza produsele din baza de date, iar cautarea, filtrarea pe categorie si sortarea actualizeaza lista;
- ☐ click pe **Detalii** deschide `/products/:id`, iar un ID inexistent (ex. `/products/9999`) afiseaza mesajul `Produsul nu exista`.

Greseli frecvente

Problema	Cauza probabila
Login-ul reuseste (`200`), dar `GET /api/auth/me` da `401`	lipseste `withCredentials: true` din instanta Axios (`src/api/client.js`)
Eroare CORS in consola browserului	serverul din Seminar 2 nu este pornit sau `origin` nu este `"http://localhost:5173"`
Pagina se reincarca la trimiterea formularului	lipseste `e.preventDefault()` in handler-ul `onSubmit`

Problema	Cauza probabila
Request-uri infinite catre `/api/products`	lipseste array-ul de dependente din `useEffect` sau este modificat un state aflat in dependente
Eroarea nu se afiseaza pe ecran	ati citit `err.message` (mesajul generic Axios) in loc de `err.response?.data?.message`
Warning in consola: `Each child in a list should have a unique "key" prop`	lipseste `key={product.id}` pe elementul randat in `.map()`
Pagina alba la pornire	ati folosit `useAuth()` in afara `<AuthProvider>`